Web-Based Library Management System

Phase 2: Progress Presentation & Design Document

SE2030 – Software Engineering | Year 2, Semester 1 (2026)

Course	SE2030 – Software Engineering
Institution	Sri Lanka Institute of Information Technology (SLIIT)
Faculty	Faculty of Computing
Year / Semester	Year 2 – Semester 1 (2026)
Group ID	Group 2
GitHub Repo	https://github.com/IT21060144/SE2030-Group2-LibrarySystem
Submission Date	30th April 2026

Group Members

Student ID	Name	Assigned Module
IT21047756	Weerasekara W.B.M.K.D	Return Management
IT21054372	Samarasinghe P.D.P	Book Management
IT21060144	Rathnayake R.M.I	Reservation Management
IT21077524	Pathirana A.P.C.E	User Management
IT21101274	Dolawatta T.Y.R	Fine Management
IT21158872	Vimukthi W.M.B.V	Borrowing Management

Table of Contents

1. Agile Sprint Summaries

Our team used the Agile method to plan and build the project. We split the work into sprints. Each sprint had a clear goal and each team member worked on their own module. By Week 10, all six modules are working. All CRUD operations, input validation and the SQLite database are complete.

Completed Sprints

Sprint 1 — Weeks 3 & 4

Goal: Set up the project and submit the proposal

– Chose the project topic: Web-Based Library Management System

– Each member was given one module to build

– Submitted the project proposal report on 8 March 2026

– Created the GitHub repository and set up React and Spring Boot

– Designed the SQLite database tables for all six modules

Sprint 2 — Weeks 5 & 6

Goal: Plan the system and set up the database

– Planned how React, Spring Boot and SQLite work together using REST APIs

– Listed all API endpoints for each module (GET, POST, PUT, DELETE)

– Created all six tables in SQLite

– Built the login page with role-based access for Admin, Librarian and Student

---

Sprint 3 — Weeks 7 & 8

Goal: Build the first three modules

– Pathirana built User Management — add, view, edit, delete users. All fields are validated.

– Samarasinghe built Book Management — add, view, edit, delete books. Checks for duplicate ISBN.

– Vimukthi built Borrowing Management — borrow a book, extend due date, cancel a borrow. Book count goes down automatically.

– All three modules were tested end to end with the database

Sprint 4 — Weeks 9 & 10 (Current Sprint)

Goal: Build the last three modules and connect everything together

– Weerasekara built Return Management — record a return, update book count, change status to RETURNED

– Rathnayake built Reservation Management — reserve a book, change or cancel it, status moves from PENDING to AVAILABLE

– Dolawatta built Fine Management — system auto-calculates fine on late return (Late Days × Rs. 10). Staff can mark it as PAID.

– All six modules now work together using the same SQLite database

– Input validation is done on every form across all six modules

– The full system works end to end — ready for the progress demo

---

Planned Sprints

Sprint 5 — Week 11

Goal: Move from SQLite to SQL Server

– Move all six database tables to Microsoft SQL Server

– Update Spring Boot connection settings for SQL Server

– Test all API endpoints again to confirm everything still works

---

Sprint 6 — Week 12

Goal: Add more features and improve the UI

– Add search and filter to the book list (by author, category, availability)

– Show borrowing history and fine summary on the student page

– Improve the UI look and make it work well on different screen sizes

– Add a reports page — overdue books, most borrowed, fine totals

---

Sprint 7 — Weeks 13 & 14

Goal: Test everything, write the report and go live

– Write unit tests for the Spring Boot backend

– Deploy the system to a server so it can be used online

– Write the final report and user guide

– Prepare and do the final presentation with a live demo

---

2. Final Use Case Diagram

The diagram below shows who uses the system and what each person can do. There are six types of users and six main modules.

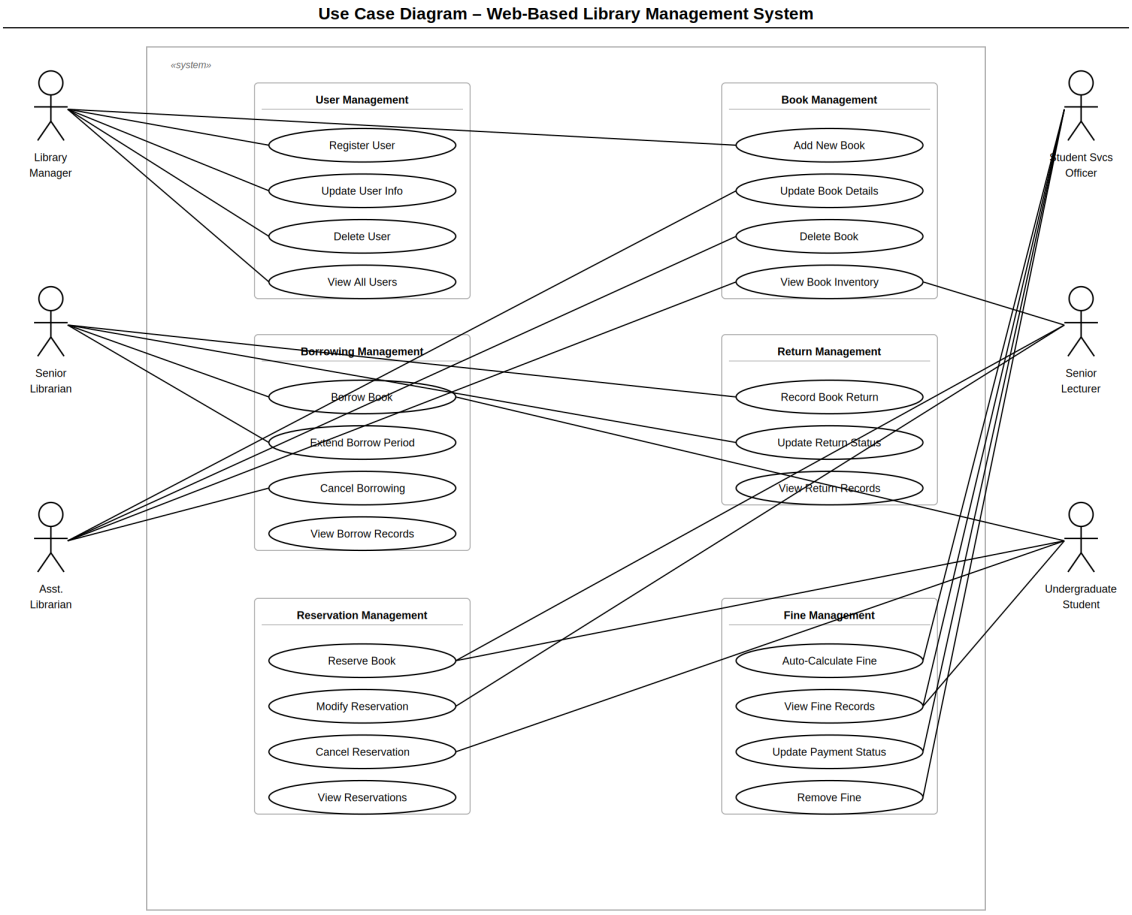

Figure 1: Use Case Diagram – Web-Based Library Management System

Actors and Their Actions

Actor	What They Can Do
Library Manager / Admin	Add, edit and remove user accounts. Assign roles to users.
Senior Librarian	Issue books to students, record returns, view borrow records.
Assistant Librarian	Add new books, update book details, remove old books.
Student Services Officer	View fines, mark fines as paid, fix any wrong fine records.
Undergraduate Students	Search for books, borrow, reserve and view their own fines.
Senior Lecturer	Search for books, borrow and reserve books for research.

3. Class Diagram

The class diagram shows the six main data classes in our system. Each class maps directly to a table in the SQLite database and a model class in Spring Boot.

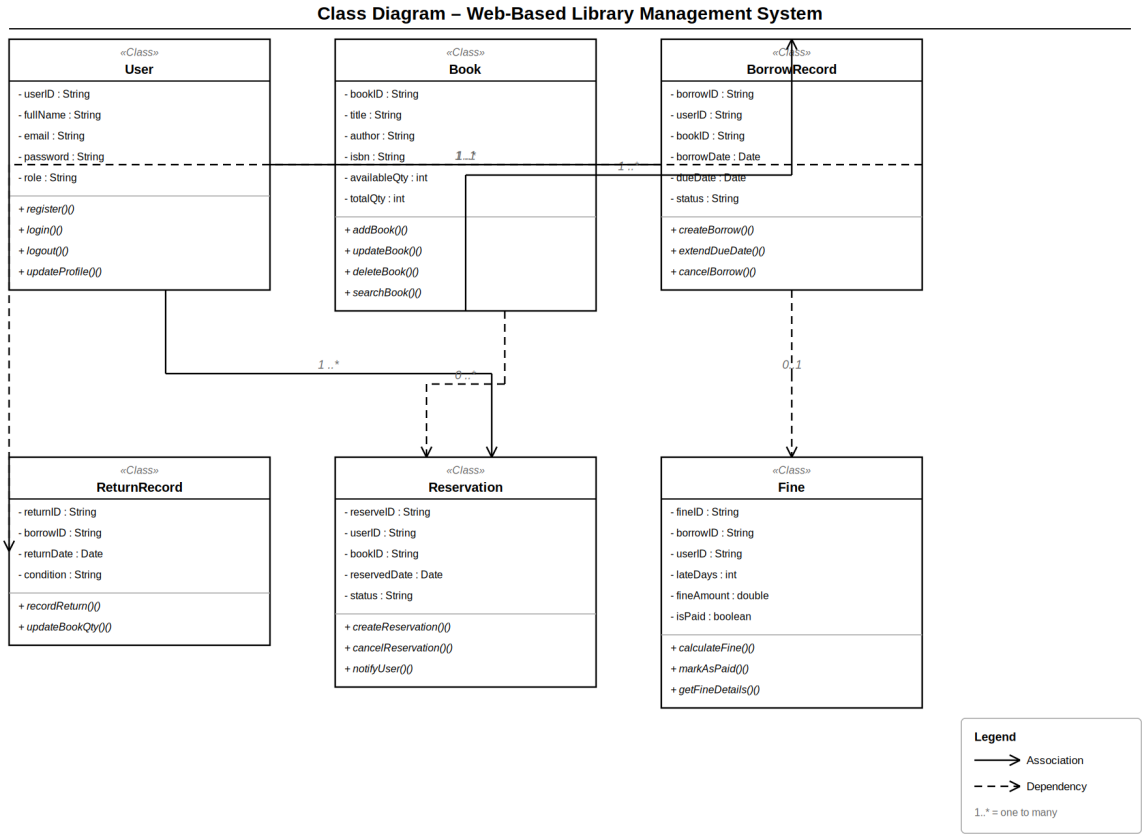

Figure 2: Class Diagram – Web-Based Library Management System

How the Classes Are Connected

Connection	Between	What It Means
One to Many	User → BorrowRecord	One user can borrow many books over time
One to Many	User → Reservation	One user can have many reservations
One to Many	Book → BorrowRecord	One book can be borrowed many times
Depends On	BorrowRecord → ReturnRecord	A return record is created from a borrow record
Depends On	BorrowRecord → Fine	A fine is created when a borrow is returned late
One to Many	User → Fine	One user can have many fine records

4. Activity Diagrams

Each diagram below shows the step-by-step flow for one module. The filled circle is the start. The bullseye is the end. The diamond is a decision point. The rounded boxes are steps.

4.1 User Management

How a new user is registered in the system.

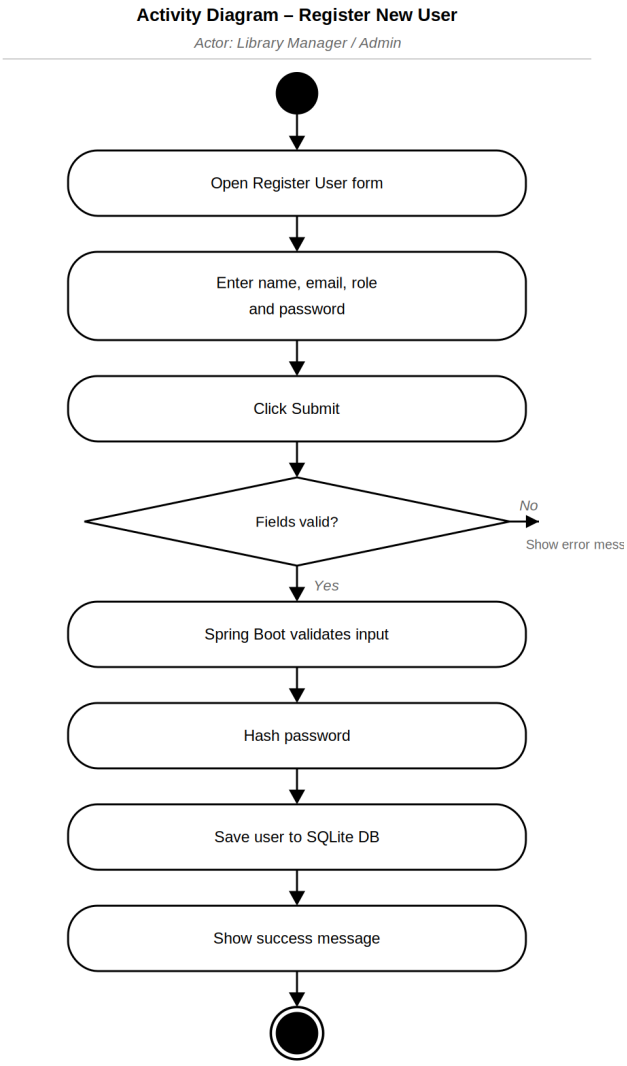

Figure 3: Activity Diagram – Register New User

4.2 Book Management

How a librarian adds a new book to the system.

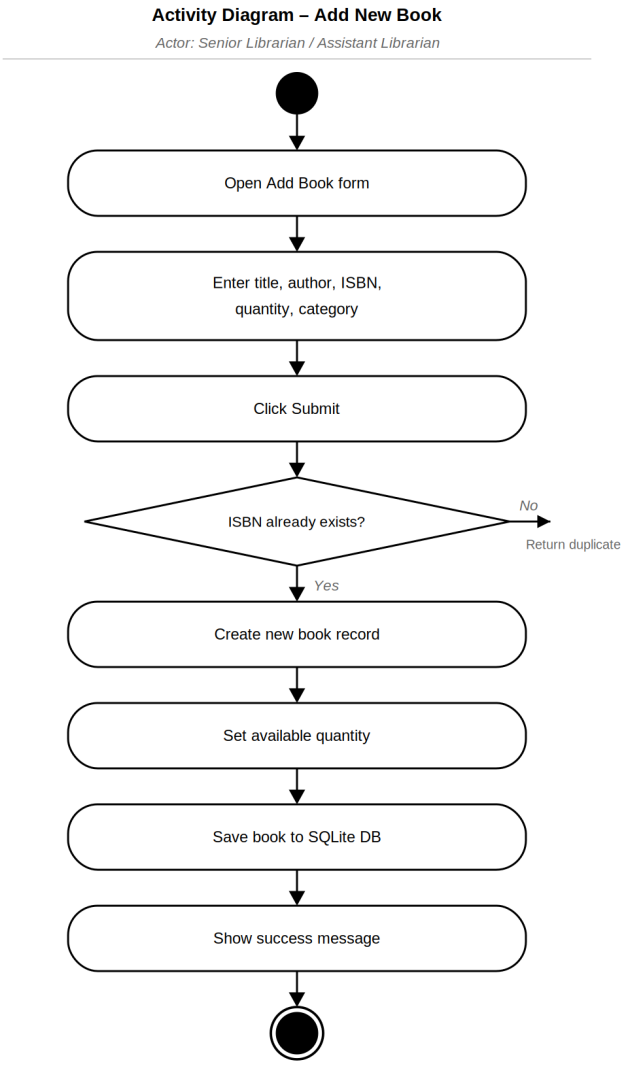

Figure 4: Activity Diagram – Add New Book

4.3 Borrowing Management

How a student borrows a book from the library.

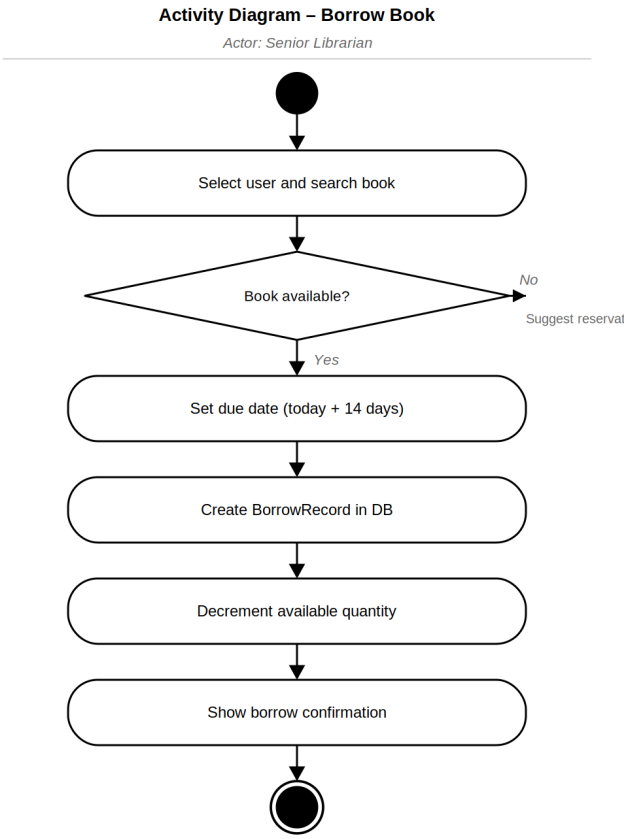

Figure 5: Activity Diagram – Borrow Book

4.4 Return Management

How a borrowed book is returned and the system records it.

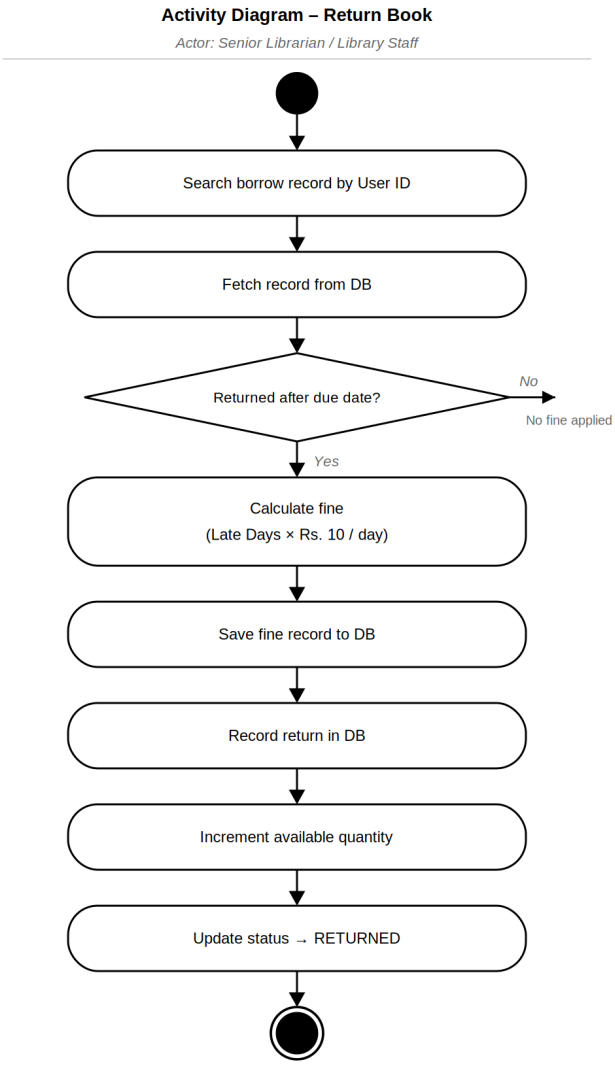

Figure 6: Activity Diagram – Return Book

4.5 Reservation Management

How a student reserves a book that is not available right now.

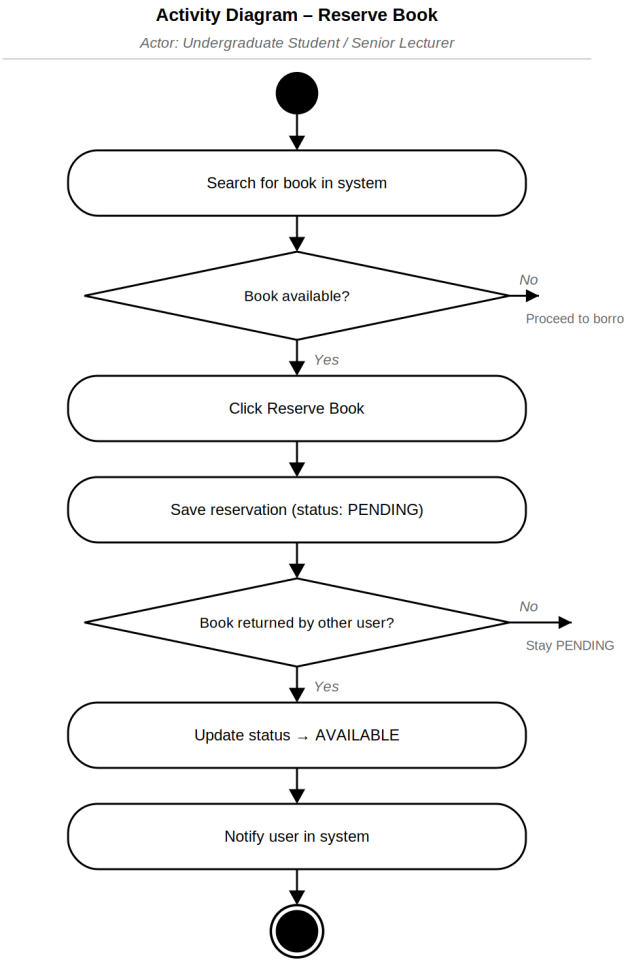

Figure 7: Activity Diagram – Reserve Book

4.6 Fine Management

How the system calculates and records a fine for a late return.

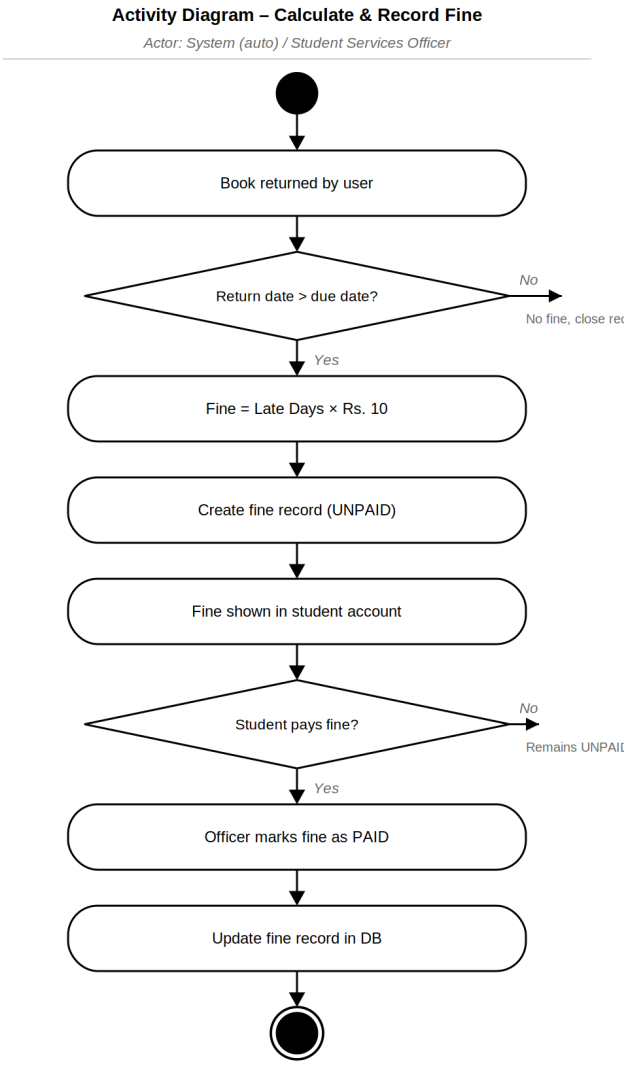

Figure 8: Activity Diagram – Calculate Fine

5. Ethical Considerations

Our system stores student information and handles fines. Because of that, we had to think carefully about doing things the right way. Below are five things we considered and what we did about each one.

1. Keeping Student Data Private

Our system saves names, emails, student IDs, borrow history and fine records. We need to make sure only the right people can see this.

Problem	Someone who should not have access could see a student's personal details or borrow history.
What we did	We used Spring Security so each user can only see their own data. Only Admins can see all records. Passwords are hashed before saving — never stored as plain text. When we move to SQL Server we will also add database-level security.

2. Telling Users What Data We Collect

Users should know what information we store about them and why.

Problem	A student might not know we are recording their borrow history and fine records.
What we did	When a user registers, we show them what data is collected. Every user can log in and view their own borrow history, reservations and fines at any time.

3. Making Sure Fines Are Correct

The system calculates fines on its own. If something goes wrong, a student could be charged the wrong amount.

Problem	A bug or wrong due date could create a wrong fine and charge a student unfairly.
What we did	The fine formula is simple and fixed: Late Days × Rs. 10 per day. The due date is always shown before a student borrows a book. Student Services Officers can check, fix or remove any fine if it is wrong.

4. Making the System Easy for Everyone

Not everyone is comfortable using web applications. The system should be easy to use for all students and staff.

Problem	Some students or staff may get confused if the system is hard to use.
What we did	We kept the React UI simple with clear labels on every form. If a user enters something wrong, an error message shows right next to that field. The system runs on any normal web browser — nothing needs to be installed.

5. Keeping the System Secure

Only people with an account should use the system, and they should only be able to do what their role allows.

Problem	Someone without an account could try to access records, or a student could try to do something only a librarian should do.
What we did	Spring Security requires login for every page. Each role has its own set of permissions — a student cannot access admin features. When a user logs out the session ends right away. For the final version we will use HTTPS to encrypt all data between the browser and server.